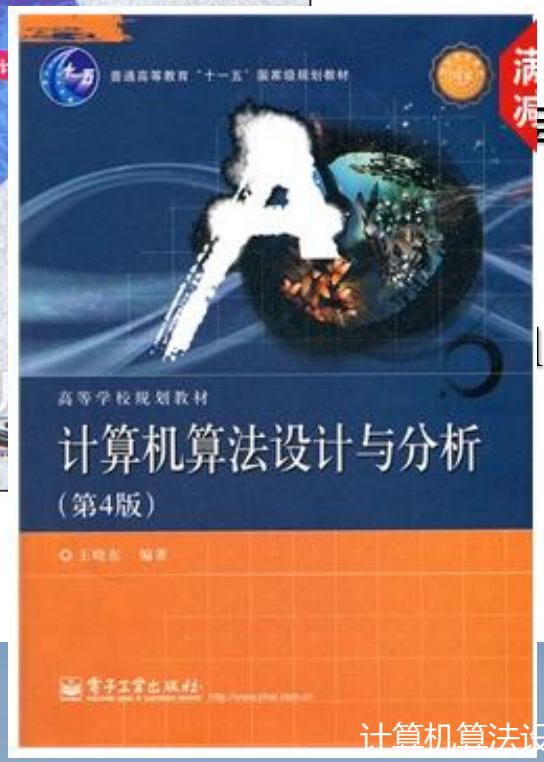
算法分析与设计
Analysis and Design of Computer Algorithms
算法概述与复杂性分析

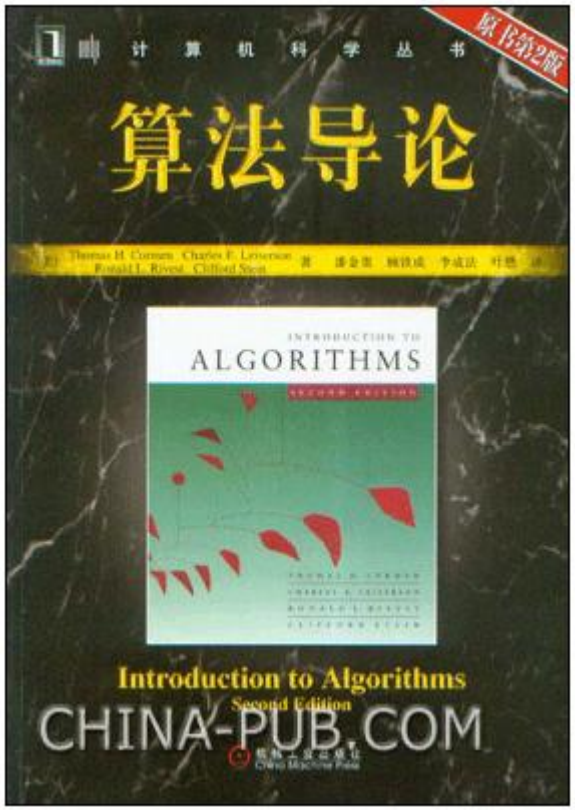
李纲
宁波大学
信息科学与工程学院

教材



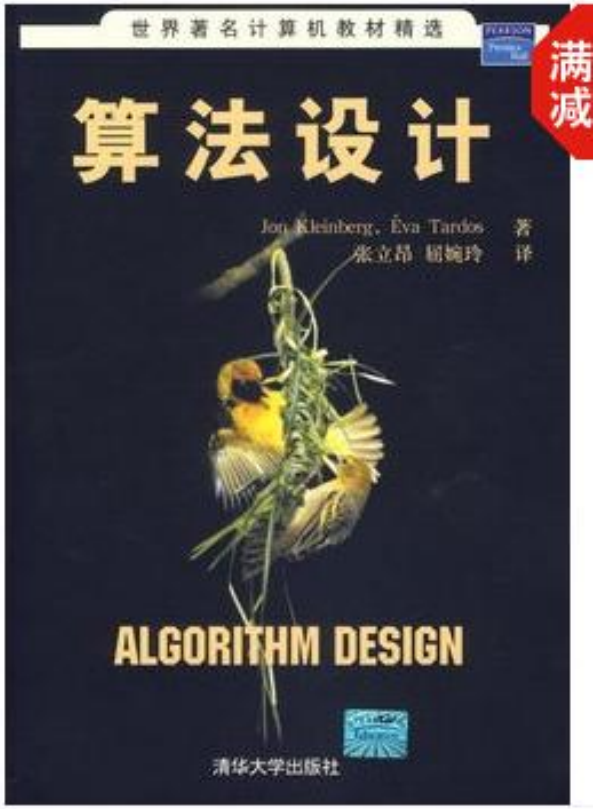
- 【作　　者】王晓东 [同作者作品]
- 【丛 书 名】普通高等教育"十一五"国家级规划教材.高等学校规划教材
- 【出 版 社】电子工业出版社　【书号】9787121042782
- 【出版日期】2007 年5月 【开 本】16开 【页 码】381　【版 次】3-1

参考书



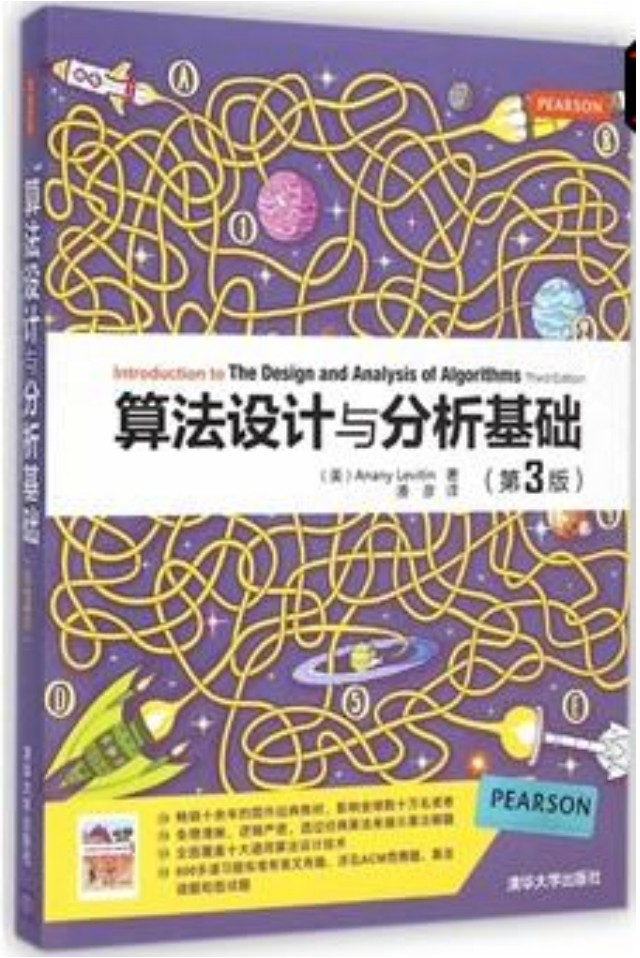
- 【原 书 名】 Introduction to Algorithms, Second Edition
- 【原出版社】 The MIT Press
- 【作　　者】 (美) Thomas H.Cormen, Charles E.Leiserson 等 [同作者作品] [作译者介绍]
- 【译　　者】 潘金贵[同译者作品] 顾铁成 李成法 叶懋
- 【丛 书 名】 计算机科学丛书
- 【出 版 社】 机械工业出版社　【书 号】 7111187776
- 【出版日期】 2006 年9月 【开 本】 16开 【页 码】 754 【版 次】 2-1

参考书



- 作　者（美）克林伯格（Kleinberg，J.），（）塔多斯（Tardos，E.）　著，张立昂，屈婉玲　译
- 出 版 社清华大学出版社
- 出版时间2007-3-1
- ＩＳＢＮ9787302143352

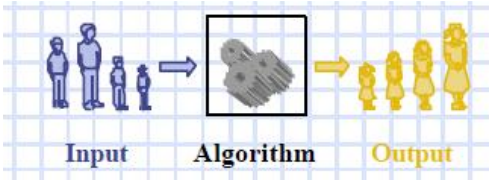
参考书



- 作　者（美）莱维汀　著，潘彦　译
- 出 版 社清华大学出版社
- 出版时间2015-2-1
- ＩＳＢＮ9787302386346
- 所属分类图书 > 计算机/网络 > 程序设计 > 其他

算法概述

- 算法是指解决问题的一种方法或一个过程。
- An **algorithm** is a step-by-step procedure for solving a problem in a finite amount of time.
- 算法是若干指令的有穷序列，满足性质：
- (1)输入：有外部提供的量作为算法的输入。一个算法有零个或多个输入，所谓零个输入是指算法本身给定了初始条件。
- (2)输出：算法产生至少一个量作为输出。



- (3)确定性：组成算法的每条指令是清晰，无歧义的。
- (4)有限性：算法中每条指令的执行次数是有限的，执行每条指令的时间也是有限的。
- (5)可行性：算法中所有的操作都必须足够基本，使算法的执行者或阅读者明确其含义以及如何执行。（能解决问题/有意义）（但可行性是针对具体的）

算法概述

- 程序是算法用某种程序设计语言的具体实现。
- 程序可以不满足算法的性质(4)。
- 例如操作系统，是一个在无限循环中执行的程序，因而不是一个算法。
- 操作系统的各种任务可看成是单独的问题，每一个问题由操作系统中的一个子程序通过特定的算法来实现。该子程序得到输出结果后便终止。

算法概述

- 如何设计算法：确定的、非确定的、哪类算法
- 如何表示算法：描述→代码，伪代码（见后续页）
- 如何确认算法：可行（见后续页）（从算法角度看正确性/有意义）
- 如何分析算法：空间、时间复杂性分析
- 如何测试算法：软件工程（黑盒、白盒）；使用公开的测试数据验证

  （从代码和输入输出角度看正确性）

算法的证明

1．通过实例　　特例

有些声明具有一般形式。“集合S中存在一个元素x，具有性质P”。为证明这条声明，只需找出某个$x \in S$，且具有性质P。同样，某些难以确认的声明也具有一般形式，“集合S中的每个元素x都具有性质P”。为了证明这个声明为假，只需找出S中的某个x不具有性质P。这样的实例称为反例（counterexample）。

示例1.11　某个教授Amongus声明每个形如2^i-1的数为素数，其中i为大于1的整数。Amongus教授是错误的。

证明　为了证明Amongus教授的声明是错误的，需要找出一个反例。幸运的是，不需看的太远，因为$2^4-1=15=3\times5$。 ■

算法的证明

奥古斯都·德·摩根首先发现了在命题逻辑中存在着下面这些关系：
非(P 且 Q) = (非 P) 或 (非 Q)
非(P 或 Q) = (非 P) 且 (非 Q)

2. “反面”进攻

另一组证明技术涉及否定的使用。有两种基本方法使用了*逆否命题法*（contrapositive）和*反证法*（contradiction）。逆否命题法就像看反面镜子。为了证明“如果p为真，那么q为真”的声明，建立“如果q不为真，那么p也不为真”的声明。逻辑上，这两个声明是相同的，但后者（称为前者的逆否命题）更容易想明白。

示例1.12 如果ab为奇数，那么a是奇数，或b是偶数。

证明 为了证明这个声明，考虑它的逆否命题，“如果a是偶数，且b是奇数，那么ab为偶数。”因此，假定对于某个整数i，$a=2i$，那么$ab=(2i)b=2(ib)$；因此，ab为偶数。 ■

除了显示逆否命题技术的使用之外，上一个例子还包括利用DeMorgan定律。这个定律可以帮助我们处理否命题。例如，形如“p或q（p or q）”的声明的否命题为“非p且非q（not p and not q）”。同样，形如“p与q（p and q）”的声明的否命题为“非p或非q（not p or not q）”。

另一种否定证明技术是*反证法*，它常常涉及利用DeMorgan定律。在利用反证法进行证明的过程中，首先建立一个真命题q，假定q为假，然后证明这个假设导致矛盾（$2\neq 2$或$1>3$）。通过这个矛盾，证明不存在使q为假的情况，于是q一定为真。当然为了得到这个结论，必须确信假设q为假之前，所有情况一致。

算法的证明

3. 归纳法

我们所做的大多数关于运行时间和空间界限的声明都与一个整型参数n（通常直观地表示问题的规模）有关。此外，这些声明中的大多数都等价于说“对于所有的$n \geq 1$”，某些声明$q(n)$为真。由于这是对一个无限的数字集合所做的声明，不能直接穷举证明它。

算法的证明

4. 循环不变式

本节最后讨论的一个证明技术是循环不变式（loop invariant）。

为证明某个声明S关于一个循环是正确的，根据一系列更小的声明$S_0, S_1, \cdots, S_k$定义S，其中：

(1) 初始声明S_0在循环开始之前为真。

(2) 如果声明S_{i-1}在第i次迭代开始之前为真，那么可以证明声明S_i在第i次迭代结束之后也为真。

(3) 最终声明S_k蕴涵着要证明的声明S为真。

插入排序，每次循环从数组A中取出第j个元素插入有序区A[1 .. j-1]，然后递增j。这样A[1 .. j-1]的有序性始终得到保持，这就是所谓的“循环不变”了。

有14张牌,两人轮流取,每次只能取1或2张,谁取到最后一张谁获胜,如果你先取,想获胜,第一次取几张?

算法的证明

有14张牌,两人轮流取,每次只能取1或2张,谁取到最后一张谁获胜,如果你先取,想获胜,第一次取几张？ （最后3张，可以看出；保持3的倍数）

你先取，然后保持3的倍数

有意识地寻找循环过程中不变的性质，就会发现：

在循环过程中,白球个数的奇偶性保持不变。要么拿2，要么不拿

每次取出的白球的个数或是零或是2。

因此,如果开始时白球的个数为奇数,那么剩下的一球为白球。如果开始时白球的个数为偶数，那么剩下的一球为黑球。

算法的证明

布口袋中有黑，白两色小球，现将手放入袋中每次摸出两个，如果两球同色就都不放回袋中，如果两球异色就将白球放回，由于每次至少减少一个，所以袋中的球必然越来越少。

现问:如果袋中最后剩下一个球，此球的颜色与开始时袋中黑、白球的个数有什么关系?

循环保持了什么特性?
2白球：白-2
2黑球：黑-2
1黑1白：黑-1

这种不依赖前面所执行的重复次数的性质称之为循环不变式。

算法的证明

横竖斜的数字相加的和相等：循环不变/每次计算原理相同

		7
10	6	

第1步，先计算10上面一格；也可以先计算7的最下面一格

伪代码

Pseudocode

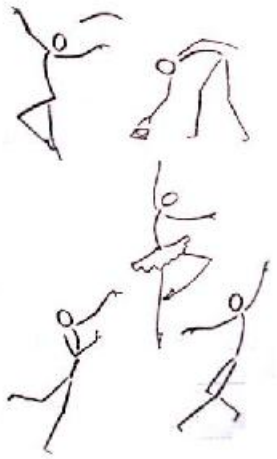
- High-level description of an algorithm
- More structured than English prose
- Less detailed than a program
- Preferred notation for describing algorithms
- Hides program design issues

注重结构，忽略细节，上层描述

Example: find max element of an array

Algorithm *arrayMax*(A, n)
  Input array A of n integers
  Output maximum element of A

  $currentMax \leftarrow A[0]$
  for $i \leftarrow 1$ **to** $n - 1$ **do**
    if $A[i] > currentMax$ **then**
      $currentMax \leftarrow A[i]$
  return $currentMax$



算法概述

- 算法(Algorithm) 的由来
- Algorithm(算法)一词本身就十分有趣。初看起来，这个词好像是某人打算要写“Logarithm”(对数)一词但却把头四个字母写的前后颠倒了。这个词一直到1957年之前在Webster's New World Dictionary(《韦氏新世界词典》)中还未出现，我们只能找到带有它的古代涵义的较老形式的“Algorism”(算术)，指的是用阿拉伯数字进行算术运算的过程。在中世纪时，珠算家用算盘进行计算，而算术家用算术进行计算。中世纪之后，对这个词的起源已经拿不准了，早期的语言学家试图推断它的来历，认为它是从把algiros(费力的)+arithmos(数字)组合起来派生而成的，但另一些人则不同意这种说法，认为这个词是从“喀斯迪尔国王 Algor”派生而来的。

算法概述

- 最后，数学史学家发现了algorism(算术)一词的真实起源：它来源于著名的 Persian Textbook，(《波斯教科书》)的作者的名字Abu Ja'far Mohammed ibn Mûsâ al-Khowârizm （约公元前825年）——从字面上看，这个名字的意思是“Ja'far 的父亲，Mohammed 和 Mûsâ 的儿子，Khowârizm 的本地人”。Khowârizm 是前苏联ХИВА(基发) 的小城镇。Al-Khowârizm 写了著名的书Kitab al jabr w'al-muqabala (《复原和化简的规则》)；另一个词，“algebra”(代数)，是从他的书的标题引出来的，尽管这本书实际上根本不是讲代数的。

算法概述

- 逐渐地，“algorism”的形式和意义就变得面目全非了。如牛津英语字典所说明 的，这个词是由于同arithmetic(算术)相混淆而形成的错拼词。由algorism又变成algorithm。一本早期的德文数学词典 Vollstandiges Mathematisches Lexicon（《数学大全辞典》），给出了Algorithmus (算法)一词的如下定义：“在这个名称之下，组合了四种类型的算术计算的概念，即加法、乘法、减法、除法”。拉顶短语algorithmus infinitesimalis (无限小方法)，在当时就用来表示Leibnitz(莱布尼兹)所发明的以无限小量进行计算的微积分方法。

算法概述

- 1950 年左右，algorithm一词经常地同欧几里德算法(Euclid's algorithm)联系在一起。这个算法就是在欧几里德的《几何原本》(Euclid's Elements ,第VII卷，命题i和ii)中所阐述的求两个数的最大公约数的过程(即辗转相除法)。

几种基本运算和几种基本语句的算法时间

- 规则(1) 赋值、比较、算术运算、逻辑运算、读写单个常量或单个变量等，只需要1个单位时间。
- 规则(2) 条件语句"if C then S_1 else S_2“，只需要$T_c+\max(T_{s1}, T_{s2})$的时间，其中T_c是计算条件表达式C需要的时间，而T_{s1}和T_{s2}分别是执行语句S_1和S_2需要的时间。
- 规则(3) 选择语句"Case A of a1:S1; a2:S2; … ;am:Sm; end"，需要max(Ts1, Ts2，…，Tsm)的时间，其中Tsi是执行语句Si所需要的时间，i=1,2,…,m。
- 在C或C++语言中执行到break位置，因此可能是$T_{Si}+\ldots+T_{Sj}$

几种基本运算和几种基本语句的算法时间

- 规则(4) 访问一个数组的单个分量或一个记录的单个域，只需要1个单位时间。
- 规则(5) 执行一个for循环语句需要的时间等于执行该循环体所需要的时间乘上循环的次数。
- 规则(6) 执行一个while循环语句"while C do S"或一个repeat循环语句" repeat S until C"，需要的时间等于计算条件表达式C需要的时间与执行循环S体需要的时间之和乘以循环的次数。与规则5不同，这里的循环次数是隐含的。

几种基本运算和几种基本语句的算法时间

- 规则(7) 对于goto语句
- 为了便于表达从循环体的中途跳转到循环体的结束或跳转到循环语句的后面语句，那么，在时间复杂性分析时可以假设它不需要任何额外的时间。
- 如果有的程序滥用了goto语句，即控制转移到前面的语句，那么情况将变得复杂起来。当这种转移造成某种循环时，只要与别的循环不交叉，保持循环的内外嵌套，则可以比照规则(1)-(6)进行分析。
- 当由于使用goto语句而使程序结构混乱时，建议改写程序然后再做分析。

几种基本运算和几种基本语句的算法时间

- 规则(8)
- 对于函数调用语句，它们需要的时间包括两部分，**一部分用于实现控制转移**，**另一部分用于执行函数本身**，这时可以根据函数调用的层次，由里向外运用规则(1)-(7)进行分析，一层一层地剥，直到计算出最外层的运行时间便是所求。(无递归调用)
- 如果函数出现**直接或间接的递归调用**，则上述由里向外逐层剥的分析行不通。这时我们可以对其中的各个递归过程(或函数)，所需要的时间假设为一个相应规模的待定函数。然后一一根据过程(或函数)的内涵建立起这些待定函数之间的递归关系得到递归方程。最后用求递归方程解的渐进阶的方法确定最坏情况下的复杂性的渐进阶。

一题两个算法

- 已知不重复且已经按从小到大排好的m个整数的数组A[1..m]（为简单起见。还设$m=2^k$，k是一个确定的非负整数）。对于给定的整数c，要求寻找一个下标i，使得A[i]=c；若找不到，则返回一个0。

两个算法的对比

- 这里问题的规模以m计算，算法重用到的

  元运算有赋值(a)、测试比较(t)和加法(s)

  等三种，它们每执行一次所需的时间常数分别为

  a, t, 和 s。

算法A：顺序查找

$$T_{max}(m)=a+2mt+(m-1)s+(m-1)a+t+a=(m+1)a+(2m+1)t+(m-1)s$$

$$T_{min}(m)=a+2t+t+a=2a+3t$$

元运算有赋值(a)、测试比较(t)和加法(s)减法(b)、除法(d)

```
Function Search (c:integer):integer;
Var j:integer;
Begin
    j:=1; {初始化} {在还没有到达A的最后一个分量且等于c的分量还没有找到时， 查找下一个分量并且进行检测}
    While (A[i]<c)and(j<m) do   j:=j+1;      //含2种运算
    if A[j]=c then search:=j {在数组A中找到等于c的分量，且此分量的下标为j}
    else Search:=0; {在数组中找不到等于c的分量}
 End;
```

在最坏的情况下，这个算法要检测A的所有m个分量才能判断在A中找不到等于c的分量。

min: 第一个就找到了 (j<m可能没执行)

算法B：二分查找

```
Function B_Search ( c: integer):integer;
Var L,U,I : integer; {U和L分别是要查找的数组的下标的上界和下界}
Found: boolean;
Begin
  L:=1; U:=m; {初始化数组下标的上下界}
  Found:=false; {当前要查找的范围是A[L]..A[U]。}        {当等于c的分量还没有找到且U>=L时，继续查找}
  While (not Found) and (U>=L) do
     Begin
       I:=(U+L) div 2; {找数组的中间分量}
       If c=A[I] then Found:=Ture else if c>A[I] then L:=I+1 else U:=I-1;
     End;
  If Found then B_Search:=1 else B_Search:=0;
End;
```

元运算有赋值(a)、测试比较(t)和加法(s)减法(b)、除法(d)

在最坏的情况下最多只要测A中的k+1（$k=\log m$，这里的log以2为底，下同）个分量，就判断c是否在A中。要求：已排序

算法B分析

- 算法B的$T_{\max}$
- 假设c在A中，即最坏情况的输入仍出现在c=A[m]时。这时，while循环的循环体恰好被执行了$\log^m+1$ 即$k+1$ 次。因为第一次执行时数据的规模为m，第二次执行时规模为m/2等等，最后一次执行时规模为1。
- 另外，这里除了用到赋值、测试和加法三种原运算外，还用到减法和除法两种元运算。补记后两种元运算每执行一次所需时间为b和d，则可以推演出：

$$T_{\max}(m) = 3(\log m)t + 2(\log m)a + 2(\log m)s + (\log m)b + (\log m)d$$
$$= (3t + 2a + 2s + b + d)\cdot \log m \qquad (2.10)$$

?

- 比较(2.7)和(2.10)，**我们看到m充分大时，在最坏情况下B的时间复杂性远小于SearchA的时间复杂性。**

复杂性的计量

- 如果分别用N、I和A来表示算法要解问题的规模、算法的输入和算法本身，用C表示算法的复杂性，那么应该有：
- $C=F(N,I,A)$
- 其中$F(N,I,A)$是N,I,A的一个确定的三元函数。如果把**时间复杂性和空间复杂性**分开，并分别用T和S来表示，那么应该有：
- $T=T(N,I,A)$ (2.1)
- $S=S(N,I,A)$ (2.2)

复杂性的计量

- 通常，我们让A(*算法本身*)隐含在复杂性函数本身当中，因而将（2.1）和（2.2）分别简写为
- $T=T(N,I)$
- $S=S(N,I)$

复杂性函数具体化

- 根据$T(N,I)$的概念，它应该是算法在一台抽象的计算机上运行所需的时间。设此抽象的计算机所提供的元运算有k种，他们分别记为$O_1,O_2,..,O_k$；再设这些元运算每执行一次所需要的时间分别为$t_1,t_2,..,t_k$。对于给定的算法A，设经过统计，用到元运算O_i的次数为e_i，i=1,2,..,k，很明显，对于每一个i，$1<=i<=k$，e_i是N和I的函数，即$e_i=e_i(N,I)$。那么有：

$$T(N,I)=\sum_{i=1}^{k} t_i \cdot e_i(N,I) \qquad (2.3)$$

- 其中t_i，i=1,2,..,k,是与N,I无关的常数。

复杂性函数具体化

- 下面只考虑三种情况的复杂性，即最坏情况、最好情况和平均情况下的时间复杂性，并分别记为$T_{\max}(N)$、$T_{\min}(N)$和$T_{\text{avg}}(N)$。
- 其中，D_N是规模为N的合法输入的集合；I^*是D_N中一个使$T(N,I^*)$达到$T_{\max}(N)$的合法输入，$I^{\sim}$是D_N中一个使$T(N,I^{\sim})$到$T_{\min}(N)$的合法输入；而$P(I)$是在算法的应用中出现输入I的概率。在数学上有：

$$T_{\max}(N)=\max_{I\in D_N}T(N,I)=\max_{I\in D_N}\sum_{i=1}^{k}t_i\cdot e_i(N,I)=\sum_{i=1}^{k}t_i\cdot e_i(N,I^*)=T(N,I^*) \tag{2.4}$$

$$T_{\min}(N)=\min_{I\in D_N}T(N,I)=\min_{I\in D_N}\sum_{i=1}^{k}t_i\cdot e_i(N,I)=\sum_{i=1}^{k}t_i\cdot e_i(N,\tilde{I})=T(N,\tilde{I}) \tag{2.5}$$

$$T_{avg}(N)=\sum_{I\in D_N}P(I)\cdot T(N,I)=\sum_{I\in D_N}P(I)\sum_{i=1}^{k}t_i\cdot e_i(N,I) \tag{2.6}$$

递归算法分析

```
算法  recursiveMax(A, n):
    输入：存储n(≥1)个整数的数组A
    输出：A中的最大元素
    if n = 1 then
        return A[0]
    return max{recursiveMax(A, n−1), A[n−1]}
```

$$T(n)=\begin{cases}3 & \text{如果} n=1\\ T(n-1)+7 & \text{其他情况}\end{cases}$$

复杂性渐近性态

- 设$T(N)$是算法A的复杂性函数。一般说来，当N单调增加且趋于∞时,$T(N)$也将单调增加趋于∞。
- 对于$T(N)$，如果存在$T'(N)$，使得当$N\to\infty$时有：
- $(T(N)-T'(N))/T(N)\to 0$
- 那么，$T'(N)$是$T(N)$当$N\to\infty$时的***渐近性态***，
- 或叫$T'(N)$为算法A当$N\to\infty$的***渐近复杂性***
- **直观上，$T'(N)$是$T(N)$中略去低阶项所留下的主项。**
- 比如当 $T(N)=3N^2+4N\log_2 N+7$
- $T'(N)=3N^2$
- 因为这时有：$$(T(N)-T'(N))/T(N)=\frac{4N\log_2 N+7}{3N^2+4N\log_2 N+7}\to 0,\qquad \text{当}N\to\infty$$
- 显然$3N^2$比$3N^2+4N\log_2 N+7$简单得多，而且直观。

渐进符号

- 算法效率的主要指标是基本操作次数的增长次数。
- 为了对这些增长次数进行比较和归类，计算机科学家们使用了3种符号：
  - O（读“O”）：　　上界
  - Ω（读"omega”）：下界
  - Θ（读"theta”）：　相等

渐进符号:O

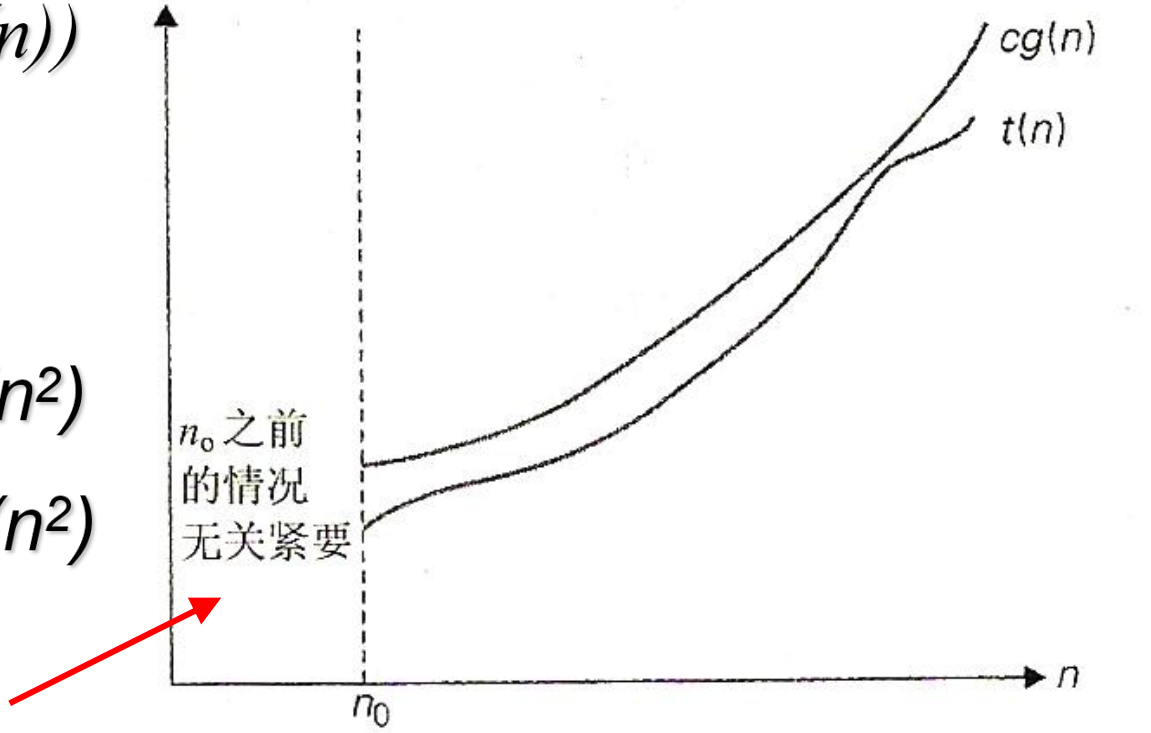
- 定义1 对于足够大的n，$t(n)$的上界由$g(n)$的常数倍来确定，即：

- 记为$t(n) \in O(g(n))$

$n \in O(n^2)$

$100n+5 \in O(n^2)$

$n(n-1)/2 \in O(n^2)$



渐进符号:O

定理 设$d(n)$、$e(n)$、$f(n)$和$g(n)$是将非负整数映射到非负实数的函数。则

(1) 如果$d(n)$是$O(f(n))$，那么对于任何常数$a>0$，$ad(n)$是$O(f(n))$。

(2) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n)+e(n)$是$O(f(n)+g(n))$。

(3) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n)\,e(n)$是$O(f(n)\,g(n))$。

(4) 如果$d(n)$是$O(f(n))$，$f(n)$是$O(g(n))$，那么$d(n)$是$O(g(n))$。

(5) 如果$f(n)$是次数为d的多项式（即，$f(n)=a_0+a_1n+\cdots+a_dn^d$），那么，$f(n)$是$O(n^d)$。

(6) 对于任意固定的$x>0$和$a>1$，n^x是$O(a^n)$。

(7) 对于任意固定的$x>0$，$\log n^x$是$O(\log n)$。

(8) 对于任意固定的常数$x>0$和$y>0$，则$\log^x n$是$O(n^y)$。

在大O符号中包含常数因子和低阶项被认为是不好的。例如，尽管称函数$2n^2$是$O(4n^2+6n\log n)$是完全正确的，但它是不合适的。我们应该试图用大O的最简单形式描述函数。

渐进符号:O

示例 $2n^3 + 4n^2 \log n$是$O(n^3)$。

证明 应用定理1.2中的规则，如下：

- $\log n$是$O(n)$（规则8）。
- $4n^2 \log n$是$O(4n^3)$（规则3）。
- $2n^3 + 4n^2 \log n$是$O(2n^3 + 4n^3)$（规则2）。
- $2n^3 + 4n^3$是$O(n^3)$（规则5或规则1）。
- $2n^3 + 4n^2 \log n$是$O(n^3)$（规则4）。

(8) 对于任意固定的常数$x > 0$和$y > 0$，则$\log^x n$是$O(n^y)$。

(1) 如果$d(n)$是$O(f(n))$，那么对于任何常数$a > 0$，$ad(n)$是$O(f(n))$。

(2) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n) + e(n)$是$O(f(n) + g(n))$。

(3) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n)\, e(n)$是$O(f(n)\, g(n))$。

(4) 如果$d(n)$是$O(f(n))$，$f(n)$是$O(g(n))$，那么$d(n)$是$O(g(n))$。

(5) 如果$f(n)$是次数为d的多项式（即，$f(n) = a_0 + a_1 n + \cdots + a_d n^d$），那么，$f(n)$是$O(n^d)$。

渐进符号: Ω

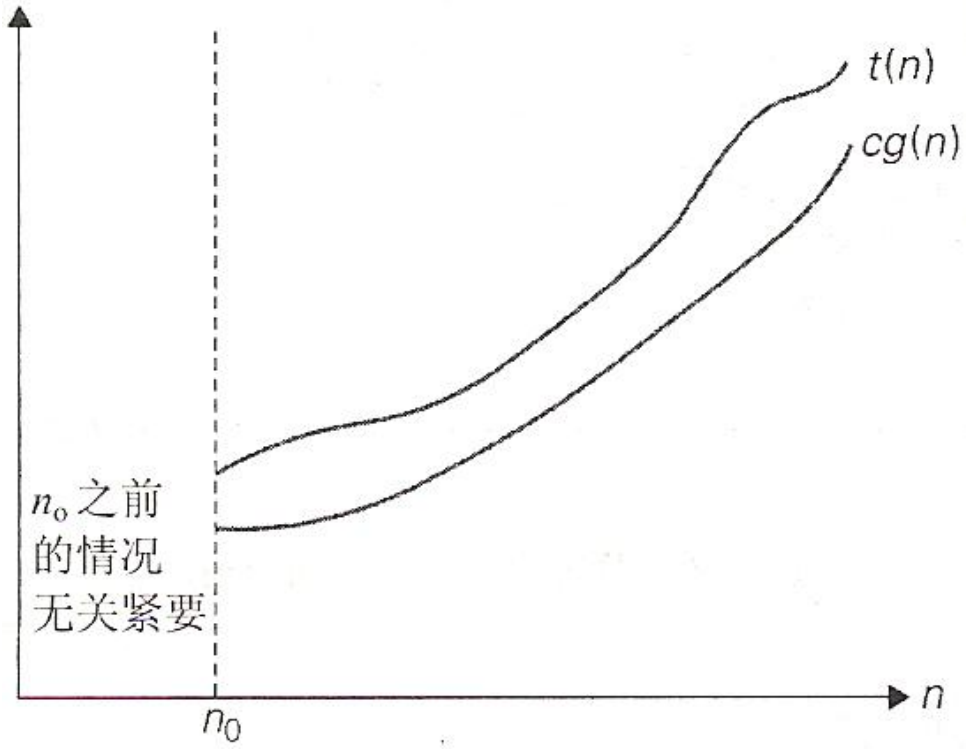
- 定义2 对于足够大的n，$t(n)$的下界由$g(n)$的常数倍来确定，即：

- 记为$t(n) \in \Omega(g(n))$

$n^3 \in \Omega(n^2)$

$n(n+1) \in \Omega(n^2)$

$4n^2+5 \in \Omega(n^2)$



渐进符号: Θ

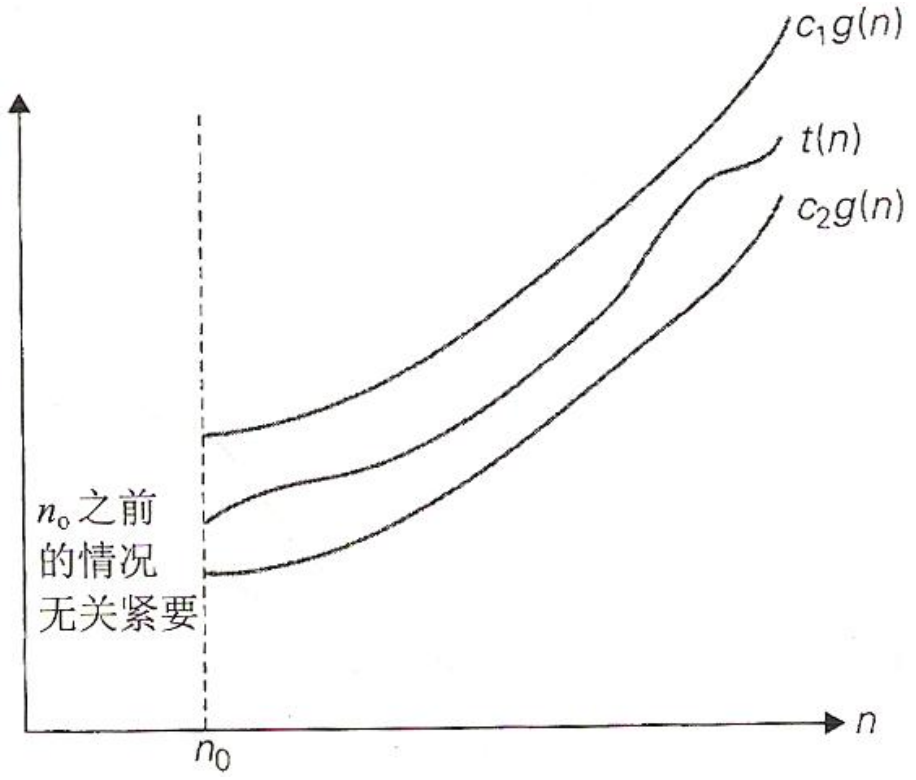
- 定义3 对于足够大的n，$t(n)$的上界和下界由$g(n)$的常数倍来确定，即：

- 记为$t(n) \in \Theta(g(n))$

$n^2+3n+2 \in \Theta(n^2)$

$n(n-1)/2 \in \Theta(n^2)$

$4n^2+5 \in \Theta(n^2)$



渐进符号

- 几个例子：
- (1)因为对所有的$N \geq 1$有$3N \leq 4N$，有$3N = O(N)$；
- (2)因为当$N \geq 1$时有$N+1024 \leq 1025N$，有$N+1024 = O(N)$；
- (3)因为当$N \geq 10$时有$2N^2+11N-10 \leq 3N^2$，有
- $2N^2+11N-10 = O(N^2)$；
- (4)因为对所有$N \geq 1$有$N^2 \leq N^3$，有$N^2 = O(N^3)$；
- (5)作为一个反例$N^3 \neq O(N^2)$。因为若不然，则存在正的常数C和自然数N_0，使得当$N \geq N_0$时有$N^3 \leq C N^2$，即$N \leq C$。显然当取$N = \max(N_0, [C]+1)$时这个不等式不成立，所以$N^3 \neq O(N^2)$。

渐进符号

- 按照大O的定义，容易证明它有如下运算规则：
- 比前面的计算规则简洁/常用

定理 设$d(n)$、$e(n)$、$f(n)$和$g(n)$是将非负整数映射到非负实数的函数。则

(1) 如果$d(n)$是$O(f(n))$，那么对于任何常数$a>0$，$ad(n)$是$O(f(n))$。

(2) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n)+e(n)$是$O(f(n)+g(n))$。

(3) 如果$d(n)$是$O(f(n))$，$e(n)$是$O(g(n))$，那么$d(n)\,e(n)$是$O(f(n)\,g(n))$。

(4) 如果$d(n)$是$O(f(n))$，$f(n)$是$O(g(n))$，那么$d(n)$是$O(g(n))$。

(5) 如果$f(n)$是次数为d的多项式（即，$f(n)=a_0+a_1n+\cdots+a_dn^d$），那么，$f(n)$是$O(n^d)$。

(6) 对于任意固定的$x>0$和$a>1$，n^x是$O(a^n)$。

(7) 对于任意固定的$x>0$，$\log n^x$是$O(\log n)$。

(8) 对于任意固定的常数$x>0$和$y>0$，则$\log^x n$是$O(n^y)$。

1. O(f)+O(g)=O(max(f,g));
2. O(f)+ O(g)=O(f +g);
3. O(f)·O(g)= O(f·g);
4. 如果g(N)= O(f(N))，则O(f)+ O(g)= O(f);
5. O(Cf(N))= O(f(N))，其中C是一个正的常数;
6. f =O(f); //是自身的上界

渐进的参考

其他：小o和小ω

可进一步参考

Michael Goodrich的算法分析与设计

例子

- 估计下面二重循环算法段在最坏情况下的时间复杂性$T(N)$的阶。

```
for i:=1 to N do
    for j:=1 to i do
        begin
            S1; S2; S3; S4;
        end;
```

直观的方法2层循环，所以是n^2
这是最大估计
如果下面的循环采用了类似2分
那么是nlogn

- 其中S_k (k=1,2,3,4)是单一的赋值语句。

例子

- 对于内循环体，显然只需$O(1)$时间。因而内循环只需：

$$\sum_{j=1}^{i} O(1) = O(1)i = O(i)$$

- 加上外循环的时间复杂性：

也可以用计算的方法算出

$$T(N) = \sum_{i=1}^{N} O(i) = O(\sum_{i=1}^{N} i) = O(\frac{N(N+1)}{2}) = O(N^2)$$

```
for i:=1 to N do
    for j:=1 to i do
        begin
            S1; S2; S3; S4;
        end;
```

直观的
这是最
如果下
那么是

算法与渐近时间复杂性的关系

算法	渐进时间复杂性 $T(N)$	在C_1上可解的规模 N_1	在C_2上可解的规模 N_2	N_1和N_2的关系
A_1	N	N_{11}	N_{21}	$N_{21}=10N_{11}$
A_2	$N\log N$	N_{12}	N_{22}	$N_{22}\approx 10N_{12}$
A_3	N^2	N_{13}	N_{23}	$N_{23}=\sqrt{10}N_{13}$
A_4	N^3	N_{14}	N_{24}	$N_{24}=\sqrt[3]{10}N_{14}$
A_5	2^N	N_{15}	N_{25}	$N_{25}=N_{15}+\log 10$
A_6	N!	N_{16}	N_{26}	$N_{26}=N_{16}+$小的常数

- 让这六种算法各在C_1和C_2两台计算机上运行，并设**计算机C_2的计算速度是计算机C_1的10倍**。
- **对于高效的算法A_1，计算机的计算速度增长10倍，可求解的规模同步增长10倍；**
- **但对于低效的算法A3，计算机的计算速度增长10倍只换取可求解的问题的规模增加log10。**

算法与渐近时间复杂性的关系

表　1秒、1分和1小时内能求解的最大问题规模，不同的运行时间以微秒度量

运行时间	最大问题规模(n)		
	1 秒	1 分	1 小时
$400n$	2 500	150 000	9 000 000
$20n\lceil\log n\rceil$	4 096	166 666	7 826 087
$2n^2$	707	5 477	42 426
n^4	31	88	244

表　在某一固定时间内，利用一台比原来快256倍的计算机，对于具有各种不同运行时间的算法，所能求解的最大问题规模将会增加。m是原最大问题规模，每一项都是m的函数

运行时间	新的最大问题规模
$400n$	$256m$
$20n\lceil\log n\rceil$	近似 $256((\log m)/(7+\log m))m$
$2n^2$	$16m$
n^4	$4m$
2^n	$m+8$

算法与渐近时间复杂性的关系

- **在讨论算法复杂性的渐近阶的重要性的同时，有两条要记住：**

1. “复杂性的渐近阶比较低的算法比复杂性的渐近阶比较高的算法有效”这个结论，**只是在问题的求解规模充分大时才成立**。比如算法A_4比A_5有效只是在$N^3<2^N$，即$N\geq c$时才成立。其中c是方程$N^3=2^N$的解。当$N<c$时，A_5反而比A_4有效。所以对于规模小的问题，不要盲目地选用复杂性阶比较低的算法。其原因一方面是如上所说，复杂性阶比较低的算法在规模小时不一定比复杂性阶比较高的算法更有效；另方面，在规模小时，决定工作效率的可能不是算法的效率而是算法的简单性，哪一种算法简单，实现起来快，就选用那一种算法。
2. 当要比较的两个算法的渐近复杂性的**阶相同时，必须进一步考察渐近复杂性表达式中常数因子**才能判别它们谁好谁差。显然常数因子小的优于常数因子大的算法。比如渐近复杂性为$N\log N/100$的算法显然比渐近复杂性为$100N\log N$的算法来得有效。

二分查找的复杂性分析：递归方法

$$T(m)=\begin{cases}1+1 & m=0\\ 1+3+2+1+1+T(0) & m=1\\ 1+3+2+1+1+3+T(m/2) & m>1\end{cases}$$

$$T(m)=\begin{cases}2 & m=0\\ 13 & m=1\\ 11+T(m/2) & m>1\end{cases}$$

- 这是一个关于T(m)的递归方程。用迭代法，容易解得：
- $T(m)=11\log m+13=\theta(\log m)$

空间复杂性$S(n)$

- 可以参照时间复杂性的分析
- 在出现函数递归调用时要考虑到其中隐含的存储空间的额外开销。

递归方程解的渐近阶的求法

- 递归方程的形式多种多样，求其解的渐近阶的方法也多种多样。这里只介绍比较实用的五种方法。

1. **代入法** 这个方法的基本步骤是先推测递归方程的显式解，然后用**数学归纳法**证明这一推测的正确性。那么，显式解的渐近阶即为所求。
2. **迭代法** 这个方法的基本步骤是通过**反复迭代**，将递归方程的右端变换成一个级数，然后求级数的和，再估计和的渐近阶；或者，不求级数的和而直接估计级数的渐近阶，从而达到对递归方程解的渐近阶的估计。
3. **套用公式法** 这个方法针对形如：$T(n)=aT(n/b)+f(n)$ 的递归方程，给出三种情况下方程解的渐近阶的三个相应估计公式供套用。

递归方程解的渐近阶的求法

- 差分方程法 有些递归方程可以看成一个差分方程，因而可以用解差分方程(初值问题)的方法来解递归方程。然后对得到的解作渐近阶的估计。
- 母函数法 这是一个有广泛适用性的方法。它不仅可以用来求解线性常系数高阶齐次和非齐次的递归方程，而且可以用来求解线性变系数高阶齐次和非齐次的递归方程，甚至可以用来求解非线性递归方程。方法的基本思想是设定递归方程解的母函数，努力建立一个关于母函数的可解方程，将其解出，然后返回递归方程的解。

递归方程解的渐近阶的求法

- 取整函数
- $\lfloor x \rfloor$：不大于x的最大整数；FLOOR(X)
- $\lceil x \rceil$：不小于x的最小整数。CEILING(X)

递归方程组解的渐进阶的求法

- 对数函数
- $\log n = \log_2 n$;
- $\lg n = \log_{10} n$;
- $\ln n = \log_e n$;
- $\log^k n = (\log n)^k$;
- $\log \log n = \log(\log n)$;
-

递归方程组解的渐进阶的求法

- for a>0 ,b>0, c>0

$$a = b^{\log_b a}$$

$$\log_b(1/a) = -\log_b a$$

$$\log_c(ab) = \log_c a + \log_c b$$

$$\log_b a = \frac{1}{\log_a b}$$

$$\log_b a^n = n \log_b a$$

$$a^{\log_b c} = c^{\log_b a}$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

递归方程组解的渐进阶的求法——代入法

- 我们要估计T(n)的上界，T(n)满足递归方程：

$$T(n) = 2T(\lfloor n/2 \rfloor) + n \qquad (6.1)$$

- 递归方程(6.1)的解的渐近阶为$O(n\log n)$。

- 我们以此为基础求解其他类似的递归方程渐进阶

递归方程组解的渐进阶的求法——代入法

- (1) 如果一个递归方程类似于你从前见过的已知其解的方程，那么推测它有类似的解是合理的。作为例子，考虑递归方程：

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n \qquad (6.3)$$

- 它在形式上与(6.1)很类似。实际上，当n充分大时

$T(\lfloor n/2 \rfloor + 17)$ 与 $T(\lfloor n/2 \rfloor)$

- 相差无几。因此可以推测(6.3)与(6.1)有类似的上界 $T(n)=O(n\log n)$。

递归方程组解的渐进阶的求法——代入法

- (2)从较宽松的界开始推测，逐步逼近精确界。
- 比如对于递归方程(6.1)，要估计其解的渐近下界。
- 由于明显地有$T(n)\geq n$，我们可以从推测$T(n)=\Omega(n)$开始
- 发现太松后，把推测的阶往上提，就可以得到
- $T(n)=\Omega(n\log n)$的精确估计

递归方程组解的渐进阶的求法——代入法

- (3)作变元的替换有时会使一个未知其解的递归方程变成类似于你曾见过的已知其解的方程，从而使得只要将变换后的方程的正确解的变元作逆变换，便可得到所需要的解。例如考虑递归方程：

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n \qquad (6.4)$$

- 看起来很复杂，因为右端变元中带根号。但是，如果作变元替换$m=\log n$，即令$n=2^m$，将其代入(6.4)，则(6.4)变成：

$$T(2^m) = 2T(\lfloor 2^{m/2} \rfloor) + m \qquad (6.5)$$

递归方程组解的渐进阶的求法——代入法

- 把m限制在正偶数集上，则(6.5)又可改写为：
- $T(2^m)=2T(2^{m/2})+m$ （注意下面是函数替换）
- 若令$S(m)=T(2^m)$，则$S(m)$满足的递归方程：
- $S(m)=2S(m/2)+m$， （m单项部分不涉及函数）
- 与(6.1)类似，因而有：
- $S(m)=O(m\log m)$，
- 进而得到$T(n)=T(2^m)=S(m)=O(m\log m)=O(\log n\log\log n)$ (6.6)

递归方程组解的渐进阶的求法——迭代法

- 即**递归树方法**
- 例子，考虑递归方程：$T(n) = 3T(\lfloor n/4 \rfloor) + n$ (6.9)
- 接连迭代二次可将右端项展开为：

$$
\begin{aligned}
T(n) &= 3T(\lfloor n/4 \rfloor) + n \\
&= n + 3(\lfloor n/4 \rfloor + 3(T(\lfloor \lfloor n/4 \rfloor /4 \rfloor) \\
&= n + 3(\lfloor n/4 \rfloor + 3(\lfloor \lfloor n/4 \rfloor /4 \rfloor + 3T(\lfloor \lfloor \lfloor n/4 \rfloor /4 \rfloor /4 \rfloor))) \qquad (6.10)
\end{aligned}
$$

- (6.10)式可化简为：

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3T(\lfloor n/4^3 \rfloor)))$$

递归方程组解的渐进阶的求法——迭代法

- 这仍然是一个递归方程，右端项还应该继续展开。容易看出，迭代 i 次后，将有

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3T(\lfloor n/4^3 \rfloor + \cdots + 3(\lfloor n/4^i \rfloor + 3T(\lfloor n/4^{i+1} \rfloor)\cdots)) \quad (6.11)$$

$$T(n) = n + 3(\lfloor n/4 \rfloor + 3(\lfloor n/4^2 \rfloor + 3(\lfloor n/4^3 \rfloor + \cdots + 3(\lfloor n/4^i \rfloor + 3T(0))\cdots))) \quad (6.13)$$

又因为$\lfloor a \rfloor \le a$，由(6.13)可得：

$$T(n) \le n + 3(n/4 + 3(n/4^2 + 3(n/4^3 + \cdots + 3(n/4^i + 3T(0))\cdots)))$$

$$= n + \frac{3}{4}n + \frac{3^2}{4^2}n + \frac{3^3}{4^3}n + \cdots + \frac{3^i}{4^i}n + 3^{i+1} \cdot T(0)$$

$$< 4n + 3^{i+1} \cdot T(0) \quad (6.14)$$

每层的量n，累计4n；
最后一项可能会有大影响（T0前面累计了很多个3）
3：子任务个数；i:迭代的层数

递归方程组解的渐进阶的求法——迭代法

- 因为$i \le \log_4 n$

$$3^{i+1} \le 3^{\log_4 n+1} = 3 * 3^{\log_4 n} = 3n^{\log_4 3}$$

$$a^{\log_b c} = c^{\log_b a}$$

$$3^{i+1} \le 3^{\log_4 n+1} = 3^{\log_3 n \cdot \log_4 3+1} = 3n^{\log_4 3}$$

?

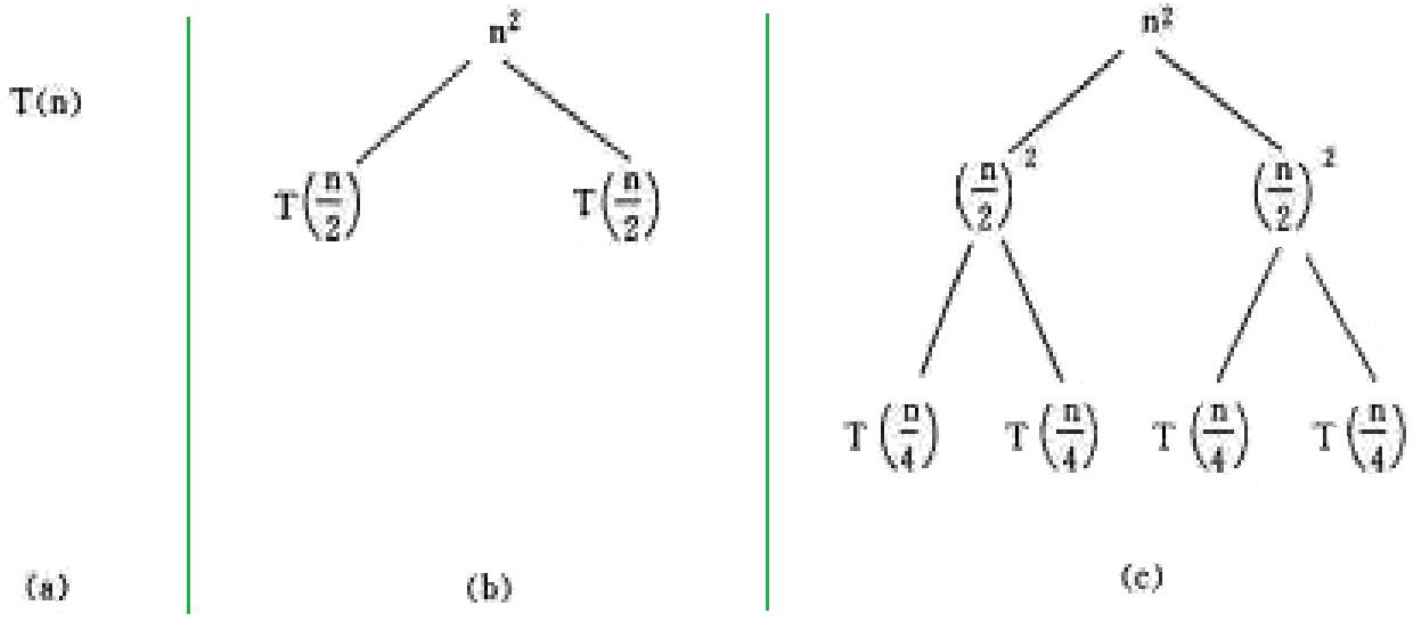
代入(6.14)得：

$$T(n) < 4n + 3n^{\log_4 3} \cdot T(0)$$

即方程(6.9)的解 $T(n)=O(n)$。

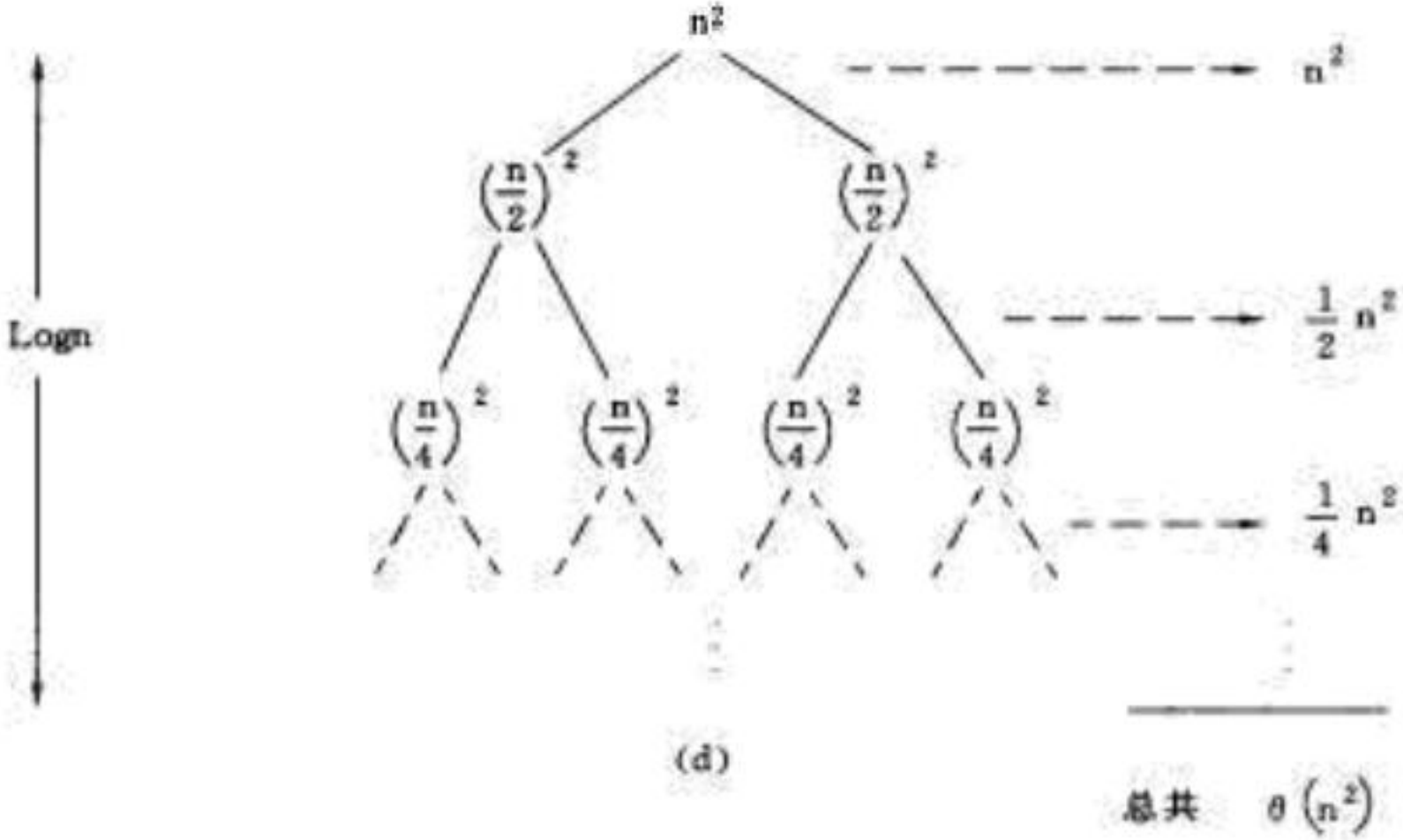
观察：当子问题个数比较多时
4表示计算量的衰减
3表示子任务个数
如果子任务个数是16，那么反而有n^2

递归方程组解的渐进阶的求法——递归树



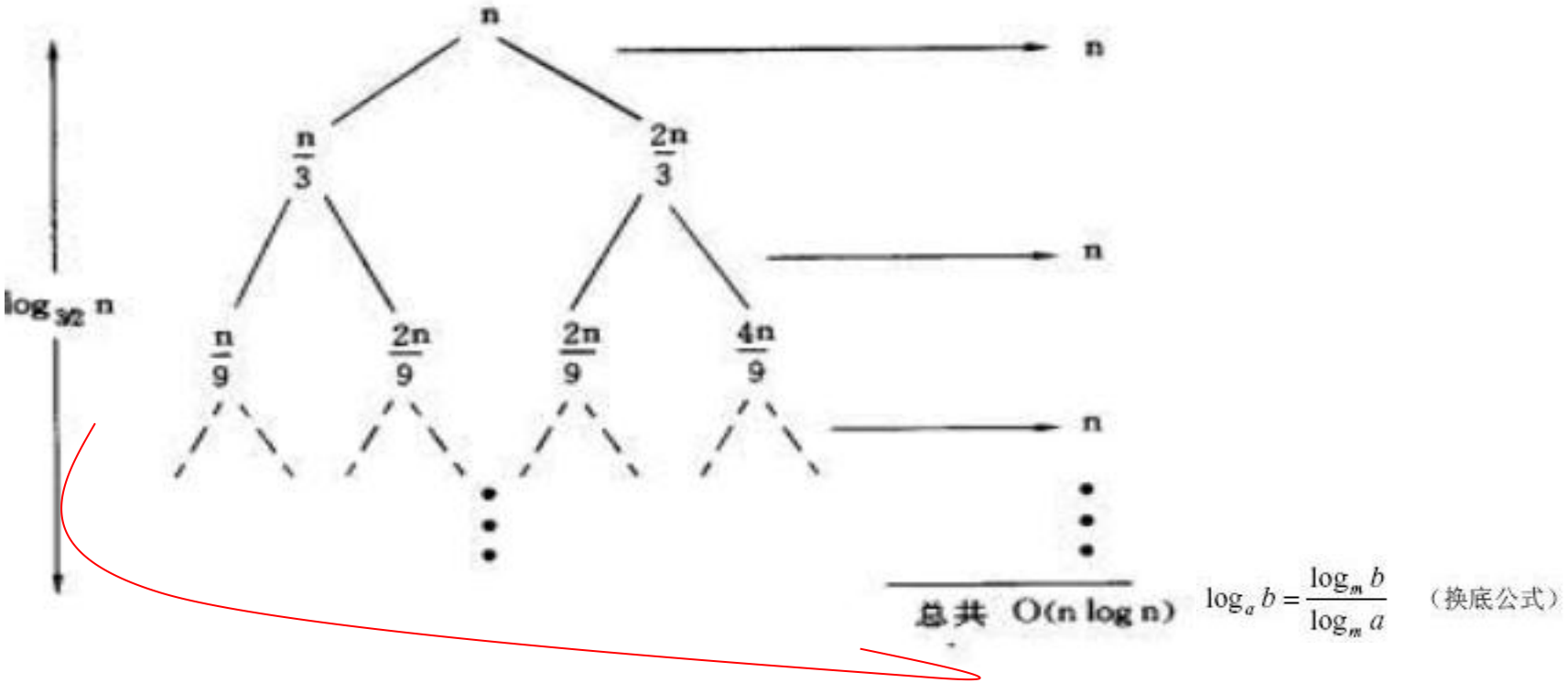
- $T(n) = 2T(n/2) + n^2$

递归方程组解的渐进阶的求法——递归树



递归方程组解的渐进阶的求法——递归树

- 再举一个例子。递归方程：
- $T(n)= T(n/3)+ T(2n/3)+n$ (6.16)



$\log_a b = \frac{\log_m b}{\log_m a}$ （换底公式）

递归方程组解的渐进阶的求法——递归树

- 设最长路径的长度为k，则应该有

$$n \to \frac{2}{3}n \to (\frac{2}{3})^2 n \to \cdots \to 1 \quad \Rightarrow \quad (\frac{2}{3})^k n = 1 \quad \Rightarrow \quad k = \log_{3/2} n$$

$$T(n) \le \sum_{i=0}^{k} n = (k+1)n = n(\log_{3/2} n + 1)$$

即 $T(n) = O(n \log n)$。

当递归树分解到1
就不能再往下分解了
也就是说到最后一层了
以此计算k层的值

递归方程组解的渐进阶的求法——套用公式法

- 该方法为估计形如：
- $T(n)=aT(n/b)+f(n)$ (6.17)
- 的递归方程解的渐近阶提供三个可套用的公式。(6.17)中的$a\geq 1$和$b\geq 1$是常数，$f(n)$是一个确定的正函数。

- (6.17)是**一类分治法**的时间复杂性所满足的递归关系，即一个规模为n的问题被分成**规模均为n/b的a个子问题**，递归地求解这a个子问题，然后通过对这a个子间题的解的综合，得到原问题的解。

递归方程组解的渐进阶的求法——套用公式法

- 该方法依据的定理：
- 设$a \geq 1$和$b \geq 1$是常数$f(n)$是定义在非负整数上的一个确定的非负函数。又设$T(n)$也是定义在非负整数上的一个非负函数，且满足递归方程(6.17)。
- 方程(6.17)中的n/b可以是$[n/b]$，也可以是n/b。那么，在$f(n)$的三类情况下，我们有$T(n)$的渐近估计式：

递归方程组解的渐进阶的求法——套用公式法

$T(n)=aT(n/b)+f(n)$

1. 若对于某常数$\varepsilon>0$，有

   $f(n)=O(n^{\log_b a-\varepsilon})$，

   则

   $T(n)=\theta(n^{\log_b a})$；

2. 若

   $f(n)=\theta(n^{\log_b a})$，

   则

   $T(n)=\theta(n^{\log_b a}\cdot\log n)$；

3. 若对其常数$\varepsilon>0$，有

   $f(n)=\Omega(n^{\log_b a+\varepsilon})$

   且对于某常数$c>1$和所有充分大的正整数n有$af(n/b)\le cf(n)$，则$T(n)=\theta(f(n))$。

递归方程组解的渐进阶的求法——套用公式法

在第一类情况下，函数$n^{\log_b a}$较大，则$T(n)=\Theta(n^{\log_b a})$；

在第二类情况下，两个函数一样大，则$T(n)=\Theta(n^{\log_b a}\log n)$，即以$n$的对数作为因子乘上$f(n)$与$T(n)$的同阶.

在第三类情况下，函数$f(n)$较大，则$T(n)=\Theta(f(n))$；

递归方程组解的渐进阶的求法——套用公式法

- **例1** 考虑 $T(n)=9T(n/3)+n$
- 有$a=9$，$b=3$，$f(n)=n$

$$n^{\log_b a}=n^{\log_3 9}=n^2$$

- 可套用第一类情况的公式，得$T(n)=\theta(n^2)$。

1. 若对于某常数$\varepsilon>0$，有

$$f(n)=O(n^{\log_b a-\varepsilon})$$

则

$$T(n)=\theta(n^{\log_b a})$$

递归方程组解的渐进阶的求法——套用公式法

- **例2** 考虑 $T(n)=T(2n/3)+1$
- 有$a=1$，$b=3/2$，$f(n)=1$

$$n^{\log_b a}=n^{\log_{3/2}1}=n^0=1=f(n)$$

- 可套用第二类情况的公式，得$T(n)=\theta(\log n)$。

2. 若

$$f(n)=\theta(n^{\log_b a})$$

则

$$T(n)=\theta(n^{\log_b a}\cdot\log n)$$

递归方程组解的渐进阶的求法——套用公式法

- **例3** 考虑 T(n)=3T(n/4)+nlogn
- 有$a=3$，$b=4$，$f(n)=n\log n$

$$n^{\log_b a}=n^{\log_4 3}=O(n^{0.793})$$

- 可套用第三类情况的公式，得$T(n)=\theta(f(n))=\theta(n\log n)$。

3. 若对其常数$\varepsilon>0$，有

$$f(n)=\Omega(n^{\log_b a+\varepsilon})$$

且对于某常数$c>1$和所有充分大的正整数n有$af(n/b)\le cf(n)$，则$T(n)=\theta(f(n))$。

递归方程组解的渐进阶的求法——套用公式法

- 有一点很重要，即要认识到上述三类情况并没有覆盖所有可能的$f(n)$。每类之间存在空隙。
- 假设用$w1$表示$n^{\log_b a}$，用$w2$表示$n^{-\varepsilon}$，用$w3$表示n^{ε}
- 在第一类情况下$f(n)$不仅必须比$w1$小，而且必须是多项式地比$w1$小，即$f(n)$必须渐近地小于$w1$与$w2$的积，ε是一个正的常数；
- 在第三类情况下$f(n)$不仅必须比$w1$大，而且必须是多项式地比$w1$大，还要满足附加的“正规性”条件：对常数c<1与所有足够大的n，$af(n/b)\leq cf(n)$。

递归方程组解的渐进阶的求法——套用公式法

- **例4** 一个本方法不适用的例子
- 考虑 $T(n)=2T(n/2)+n\log n$
- 有$a=2$，$b=2$，$f(n)=n\log n$

$n^{\log_b a}=n$

虽然$f(n)$渐近地大于$n^{\log_a b}$，但$f(n)$并不是多项式地大于$n^{\log_a b}$，

$$f(n)/n^{\log_b a+\varepsilon}=n\log n/n^{1+\varepsilon}=n^{-\varepsilon}\log n\to 0$$

- 即f(n)在第二类情况与第三类情况的间隙里，本法无效。

- f(x)多项式大于g(x)：存在实数e>0,使得f(x)>g(x)*n^e
- 两个函数的商大于一个多项式

递归方程组解的渐进阶的求法——差分方程法

- 考虑形如：
- $T(n)=c_1T(n\text{-}1)+c_2T(n\text{-}2)+\ldots+c_kT(n\text{-}k)+f(n)$，$n\geq k$　　(6.18)
- 的递归方程。

递归方程组解的渐进阶的求法——母函数法

- 参考组合数学

空间与时间的关系：LeeCode两数之和

- 方法一：暴力法
- 遍历每个元素x，查找是否存在一个值与target-x相等的目标元素。
- 时间复杂度：$O(n^2)$
- 空间复杂度：$O(1)$

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            for(int j=i+1;j<nums.size();j++){
                if(nums[i]+nums[j]==target)
                    return vector<int>{i,j};
            }
        }
        return vector<int>{};
    }
};
```

空间与时间的关系：LeeCode两数之和

- 方法二：两遍哈希表
- 时间复杂度：O(n)
- 空间复杂度：O(n)。

```
class Solution{
public:
        vector<int> twoSum(vector<int>& nums, int target){
            map<int,int> hash;
            for(int i=0;i<nums.size();i++) hash[nums[i]]=i;
            for(int i=0;i<nums.size();i++){
                int t=target-nums[i];
                if(hash.find(t)!=hash.end()&&hash[t]!=i) return vector<int>{i,hash.find(t)->second};
            }
            return vector<int>();
        }
};
```

空间与时间的关系：LeeCode两数之和

- 方法三：两遍哈希表：是否能跟前面匹配；如否，插入，供后面匹配
- 时间复杂度：O(n)
- 空间复杂度：O(n)。

```
class Solution{
public:
        vector<int> twoSum(vector<int>& nums, int target){
            map<int,int> hash;
            for(int i=0;i<nums.size();i++){
                int t= target-nums[i];
                if(hash.find(t)!=hash.end())  return vector<int>{hash.find(t)->second,i};
                hash[nums[i]]=i;
            }
        return vector<int> {};
        }
};
```

递归方程组解的渐进阶的求法

- 讨论：
- 100万个数，求第k大的数（后面讲）
- 100万个数，求第k1-k2小的数（比如3-7）

- 从算法角度，如果一个机器的CPU提速了5倍，对用户来说，其体验有何改变？

- 有T(n) =2T(n/3)+n，则该算法的复杂度是O(?)